

codex-windows / 019f331e-51d8-7972-859d-2860dc5377f3

Metadata

```
- Source: `codex-windows`  
- Kind: `codex`  
- Source file: `C:\Users\avidu\.codex\sessions\2026\07\05\rollout-2026-07-05T22-00-52-019f331e-51d8-7972-859d-2860dc5377f3.jsonl`  
- SHA-256: `2fb65cac22ca4b9ec58490c664aa7a432087c341e3b4f2f279c4513332f322a3`  
- Source modified: `2026-07-05T16:48:14+00:00`  
- Imported at: `2026-07-05T16:48:16+00:00`  
- cli_version: `0.142.5`  
- cwd: `C:\Users\avidu\Documents\Codex\2026-07-05\can`  
- model_provider: `openai`  
- session_id: `019f331e-51d8-7972-859d-2860dc5377f3`  
- source: `vscode`
```

Transcript

1. developer (2026-07-05T16:31:47.924Z)

```
<permissions instructions>  
Filesystem sandboxing defines which files can be read or written. `sandbox_mode` is `danger-  
full-access`: No filesystem sandboxing - all commands are permitted. Network access is enabled.  
Approval policy is currently never. Do not provide the `sandbox_permissions` for any reason,  
commands will be rejected.  
</permissions instructions>  
<app-context>
```

Codex desktop context

- You are running inside the Codex (desktop) app, which allows some additional features not available in the CLI alone:

Images/Visuals/Files

- In the app, the model can display images and videos using standard Markdown image syntax: `![alt](url)`
- When sending or referencing a local image or video, always use an absolute filesystem path in the Markdown image tag (e.g., `![alt](/absolute/path.png)`); relative paths and plain text will not render the media.
- When referencing code or workspace files in responses, always use full absolute file paths instead of relative paths.
- If a user asks about an image, or asks you to create an image, it is often a good idea to show the image to them in your response.
- Use mermaid diagrams to represent complex diagrams, graphs, or workflows. Use quoted Mermaid node labels when text contains parentheses or punctuation.
- Return web URLs as Markdown links (e.g., `[label](https://example.com)`).

Workspace Dependencies

- For sheets, slides, and documents, call ``load_workspace_dependencies`` to find the bundled runtime and libraries.

Automations

- This app supports recurring automations, reminders, monitors, follow-ups, and thread wakeups. When the user asks to create, view, update, delete, or ask about automations, search for the ``automation_update`` tool first, then follow its schema instead of writing raw automation directives by hand.

- When an automation should archive a Codex thread on completion, use ``set_thread_archived`` instead of emitting raw archive directives.

Thread Coordination

- When the user asks to create, fork, inspect, continue, hand off, pin, archive, rename, or otherwise manage Codex threads, search for the relevant thread tool first: ``create_thread``, ``fork_thread``, ``list_threads``, ``read_thread``, ``send_message_to_thread``, ``handoff_thread``,

``set_thread_pinned`, `set_thread_archived`, or `set_thread_title`.`

- Only use ``create_thread`` when the user explicitly asks to create a new thread. Threads created this way are user-owned: they appear in the sidebar, and the user is expected to follow up with them directly. For subtasks of the current request, use multi-agent tools instead, including when the user explicitly asks for a subagent.
- After a successful ``create_thread`` call, emit ``::created-thread{threadId="..."}`` for a created thread or ``::created-thread{pendingWorktreeId="..."}`` for queued worktree setup on its own line in your final response.

Inline Code Comments

- Use the `::code-comment{...}` directive when you need to attach feedback directly to specific code lines.
- Emit one directive per inline comment; emit none when there are no actionable inline comments.
- Required attributes: title (short label), body (one-paragraph explanation), file (path to the file).
- Optional attributes: start, end (1-based line numbers), priority (0-3).
- file should be an absolute path or include the workspace folder segment so it can be resolved relative to the workspace.
- Keep line ranges tight; end defaults to start.
- Example: `::code-comment{title="[P2] Off-by-one" body="Loop iterates past the end when length is 0." file="/path/to/foo.ts" start=10 end=11 priority=2}`

</app-context>

Projectless Chat

This projectless thread starts in a generated directory under the user's Documents/Codex folder. Prefer answering inline in chat unless using local files would make the result more useful. Use work/ for intermediate files, scratch analysis, scripts, drafts, and temporary assets. Use C:\Users\avidu\Documents\Codex\2026-07-05\can\outputs only for user-facing deliverables that should appear as outputs.

When referring to saved deliverables in the final response, link only files from

C:\Users\avidu\Documents\Codex\2026-07-05\can\outputs.

Do not write directly in the home directory unless the user explicitly asks.

<collaboration_mode># Collaboration Mode: Default

You are now in Default mode. Any previous instructions for other modes (e.g. Plan mode) are no longer active.

Your active mode changes only when new developer instructions with a different ``<collaboration_mode>...</collaboration_mode>`` change it; user requests or tool descriptions do not change mode by themselves. Known mode names are Default and Plan.

request_user_input availability

Use the ``request_user_input`` tool only when it is listed in the available tools for this turn.

In Default mode, strongly prefer making reasonable assumptions and executing the user's request rather than stopping to ask questions. If you absolutely must ask a question because the answer cannot be discovered from local context and a reasonable assumption would be risky, ask the user directly with a concise plain-text question. Never write a multiple choice question as a textual assistant message.

</collaboration_mode>

<apps_instructions>

Apps (Connectors)

Apps (Connectors) can be explicitly triggered in user messages in the format ``[$app-name]{app://{connector_id}}``. Apps can also be implicitly triggered as long as the context suggests usage of available apps.

An app is equivalent to a set of MCP tools within the ``codex_apps`` MCP.

An installed app's MCP tools are either provided to you already, or can be lazy-loaded through the ``tool_search`` tool. If ``tool_search`` is available, the apps that are searchable by ``tools_search`` will be listed by it.

Do not additionally call `list_mcp_resources` or `list_mcp_resource_templates` for apps.

</apps_instructions>

<skills_instructions>

Skills

A skill is a set of local instructions to follow that is stored in a `SKILL.md` file. Below is the list of skills that can be used. Each entry includes a name, description, and a short path that can be expanded into an absolute path using the skill roots table.

Skill roots

```
- `r0` = `C:/Users/avidu/.agents/skills`
- `r1` = `C:/Users/avidu/.codex/skills/.system`
- `r2` = `C:/Users/avidu/.codex/plugins/cache/claude-cowork/anthropic-skills/1.0.0/skills`
- `r3` = `C:/Users/avidu/.codex/plugins/cache/claude-cowork/cowork-plugin-management/0.2.2/skills`
- `r4` = `C:/Users/avidu/.codex/plugins/cache/claude-cowork/design/1.2.0/skills`
- `r5` = `C:/Users/avidu/.codex/plugins/cache/claude-cowork/engineering/1.2.0/skills`
- `r6` = `C:/Users/avidu/.codex/plugins/cache/claude-cowork/productivity/1.3.0/skills`
- `r7` = `C:/Users/avidu/.codex/plugins/cache/openai-bundled`
- `r8` = `C:/Users/avidu/.codex/plugins/cache/openai-curated-remote/github/0.1.5/skills`
- `r9` = `C:/Users/avidu/.codex/plugins/cache/openai-curated-remote/gmail/0.1.3/skills`
- `r10` = `C:/Users/avidu/.codex/plugins/cache/openai-curated-remote/google-calendar/1.2.3/skills`
- `r11` = `C:/Users/avidu/.codex/plugins/cache/openai-curated-remote/google-drive/0.1.7/skills`
- `r12` = `C:/Users/avidu/.codex/plugins/cache/openai-curated-remote/slack/0.1.2/skills`
- `r13` = `C:/Users/avidu/.codex/plugins/cache/openai-primary-runtime`
```

Available skills

- **imagegen**: Generate or edit raster images when the task benefits from AI-created bitmap visuals such as photos, illustrations, textures, sprites, mockups, or transparent-background cutouts. Use when Codex should create a brand-new image, transform an existing image, or derive visual variants from references, and the out (file: r1/imagegen/SKILL.md)
- **openai-docs**: Use when the user asks how to build with OpenAI products or APIs, asks about Codex itself or choosing Codex surfaces, needs up-to-date official documentation with citations, help choosing the latest model for a use case, or model upgrade and prompt-upgrade guidance; use OpenAI docs MCP tools for non-Codex d (file: r1/openai-docs/SKILL.md)
- **plugin-creator**: Create and scaffold plugin directories for Codex with a required `.codex-plugin/plugin.json`, optional plugin folders/files, valid manifest defaults, and personal-marketplace entries by default. Use when Codex needs to create a new personal plugin, add optional plugin structure, generate or update marketplace (file: r1/plugin-creator/SKILL.md)
- **skill-creator**: Guide for creating effective skills. This skill should be used when users want to create a new skill (or update an existing skill) that extends Codex's capabilities with specialized knowledge, workflows, or tool integrations. (file: r1/skill-creator/SKILL.md)
- **skill-installer**: Install Codex skills into \$CODEX_HOME/skills from a curated list or a GitHub repo path. Use when a user asks to list installable skills, install a curated skill, or install a skill from another repo (including private repos). (file: r1/skill-installer/SKILL.md)
- **anthropic-skills:consolidate-memory**: Reflective pass over your memory files ? merge duplicates, fix stale facts, prune the index. (file: r2/consolidate-memory/SKILL.md)
- **anthropic-skills:doc-coauthoring**: Guide users through a structured workflow for co-authoring documentation. Use when user wants to write documentation, proposals, technical specs, decision docs, or similar structured content. This workflow helps users efficiently transfer context, refine content through iteration, and verify the doc works for (file: r2/doc-coauthoring/SKILL.md)
- **anthropic-skills:docx**: Use this skill whenever the user wants to create, read, edit, or manipulate Word documents (.docx files). Triggers include: any mention of 'Word doc', 'word document', '.docx', or requests to produce professional documents with formatting like tables of contents, headings, page numbers, or letterheads. Also (file: r2/docx/SKILL.md)
- **anthropic-skills:new-session-hello**: I want claude to read the github repos linked and have some context when I start a new coding session (file: r2/new-session-hello/SKILL.md)
- **anthropic-skills:pdf**: Use this skill whenever the user wants to do anything with PDF files. This includes reading or extracting text/tables from PDFs, combining or merging multiple PDFs into one, splitting PDFs apart, rotating pages, adding watermarks, creating new PDFs, filling PDF forms, encrypting/decrypting PDFs, extracting ima (file: r2/pdf/SKILL.md)
- **anthropic-skills:pptx**: Use this skill any time a .pptx file is involved in any way ? as input, output, or both. This includes: creating slide decks, pitch decks, or presentations; reading, parsing, or extracting text from any .pptx file (even if the extracted content will be used elsewhere, like in an email or summary); editing, mod (file: r2/pptx/SKILL.md)
- **anthropic-skills:schedule**: Create or update a scheduled task that runs automatically. Use when the user says things like "every day", "each morning", "remind me in an hour", "run this at noon", or wants to reschedule an existing task. (file: r2/schedule/SKILL.md)

- anthropic-skills:setup-cowork: Guided Cowork setup ? install role-matched plugins, connect your tools, try a skill. (file: r2/setup-cowork/SKILL.md)
- anthropic-skills:skill-creator: Create new skills, modify and improve existing skills, and measure skill performance. Use when users want to create a skill from scratch, edit, or optimize an existing skill, run evals to test a skill, benchmark skill performance with variance analysis, or optimize a skill's description for better triggering a (file: r2/skill-creator/SKILL.md)
- anthropic-skills:xlsx: Use this skill any time a spreadsheet file is the primary input or output. This means any task where the user wants to: open, read, edit, or fix an existing .xlsx, .xlsm, .csv, or .tsv file (e.g., adding columns, computing formulas, formatting, charting, cleaning messy data); create a new spreadsheet from sc (file: r2/xlsx/SKILL.md)
- browser:control-in-app-browser: Control the in-app Browser. Use to open, navigate, inspect, test, click, type, screenshot, or verify local targets such as localhost, 127.0.0.1, ::1, file://, the current in-app browser tab, and websites shown side by side inside Codex. (file: r7/browser/26.623.101652/skills/control-in-app-browser/SKILL.md)
- chrome:control-chrome: Control the user's Chrome browser for tasks that depend on existing Chrome state: tabs, logged-in sessions, or extensions. Prefer purpose-built connectors, APIs, or CLIs when available. (file: r7/chrome/26.623.101652/skills/control-chrome/SKILL.md)
- computer-use:computer-use: Control Windows apps from Codex (file: r7/computer-use/26.623.101652/skills/computer-use/SKILL.md)
- cowork-plugin-management:cowork-plugin-customizer: Customize a Claude Code plugin for a specific organization's tools and workflows. Use when: customize plugin, set up plugin, configure plugin, tailor plugin, adjust plugin settings, customize plugin connectors, customize plugin skill, tweak plugin, modify plugin configuration. (file: r3/cowork-plugin-customizer/SKILL.md)
- cowork-plugin-management:create-cowork-plugin: Guide users through creating a new plugin from scratch in a cowork session. Use when users want to create a plugin, build a plugin, make a new plugin, develop a plugin, scaffold a plugin, start a plugin from scratch, or design a plugin. This skill requires Cowork mode with access to the outputs directory for (file: r3/create-cowork-plugin/SKILL.md)
- design:accessibility-review: Run a WCAG 2.1 AA accessibility audit on a design or page. Trigger with "audit accessibility", "check a11y", "is this accessible?", or when reviewing a design for color contrast, keyboard navigation, touch target size, or screen reader behavior before handoff. (file: r4/accessibility-review/SKILL.md)
- design:design-critique: Get structured design feedback on usability, hierarchy, and consistency. Trigger with "review this design", "critique this mockup", "what do you think of this screen?", or when sharing a Figma link or screenshot for feedback at any stage from exploration to final polish. (file: r4/design-critique/SKILL.md)
- design:design-handoff: Generate developer handoff specs from a design. Use when a design is ready for engineering and needs a spec sheet covering layout, design tokens, component props, interaction states, responsive breakpoints, edge cases, and animation details. (file: r4/design-handoff/SKILL.md)
- design:design-system: Audit, document, or extend your design system. Use when checking for naming inconsistencies or hardcoded values across components, writing documentation for a component's variants, states, and accessibility notes, or designing a new pattern that fits the existing system. (file: r4/design-system/SKILL.md)
- design:research-synthesis: Synthesize user research into themes, insights, and recommendations. Use when you have interview transcripts, survey results, usability test notes, support tickets, or NPS responses that need to be distilled into patterns, user segments, and prioritized next steps. (file: r4/research-synthesis/SKILL.md)
- design:user-research: Plan, conduct, and synthesize user research. Trigger with "user research plan", "interview guide", "usability test", "survey design", "research questions", or when the user needs help with any aspect of understanding their users through research. (file: r4/user-research/SKILL.md)
- design:ux-copy: Write or review UX copy ? microcopy, error messages, empty states, CTAs. Trigger with "write copy for", "what should this button say?", "review this error message", or when naming a CTA, wording a confirmation dialog, filling an empty state, or writing onboarding text. (file: r4/ux-copy/SKILL.md)
- documents:documents: Create, edit, redline, and comment on `.docx`, Word, and Google Docs-targeted document artifacts inside the container, with a strict render-and-verify workflow. Use `render\_docx.py` to generate page PNGs (and optional PDF) for visual QA, then iterate until layout is flawless before delivering the final doc (file: r13/documents/26.630.12135/skills/documents/SKILL.md)
- engineering:architecture: Create or evaluate an architecture decision record (ADR). Use when choosing between technologies (e.g., Kafka vs SQS), documenting a design decision with trade-

offs and consequences, reviewing a system design proposal, or designing a new component from requirements and constraints. (file: r5/architecture/SKILL.md)

- engineering:code-review: Review code changes for security, performance, and correctness. Trigger with a PR URL or diff, "review this before I merge", "is this code safe?", or when checking a change for N+1 queries, injection risks, missing edge cases, or error handling gaps. (file: r5/code-review/SKILL.md)
- engineering:debug: Structured debugging session ? reproduce, isolate, diagnose, and fix. Trigger with an error message or stack trace, "this works in staging but not prod", "something broke after the deploy", or when behavior diverges from expected and the cause isn't obvious. (file: r5/debug/SKILL.md)
- engineering:deploy-checklist: Pre-deployment verification checklist. Use when about to ship a release, deploying a change with database migrations or feature flags, verifying CI status and approvals before going to production, or documenting rollback triggers ahead of time. (file: r5/deploy-checklist/SKILL.md)
- engineering:documentation: Write and maintain technical documentation. Trigger with "write docs for", "document this", "create a README", "write a runbook", "onboarding guide", or when the user needs help with any form of technical writing ? API docs, architecture docs, or operational runbooks. (file: r5/documentation/SKILL.md)
- engineering:incident-response: Run an incident response workflow ? triage, communicate, and write postmortem. Trigger with "we have an incident", "production is down", an alert that needs severity assessment, a status update mid-incident, or when writing a blameless postmortem after resolution. (file: r5/incident-response/SKILL.md)
- engineering:standup: Generate a standup update from recent activity. Use when preparing for daily standup, summarizing yesterday's commits and PRs and ticket moves, formatting work into yesterday/today/blockers, or structuring a few rough notes into a shareable update. (file: r5/standup/SKILL.md)
- engineering:system-design: Design systems, services, and architectures. Trigger with "design a system for", "how should we architect", "system design for", "what's the right architecture for", or when the user needs help with API design, data modeling, or service boundaries. (file: r5/system-design/SKILL.md)
- engineering:tech-debt: Identify, categorize, and prioritize technical debt. Trigger with "tech debt", "technical debt audit", "what should we refactor", "code health", or when the user asks about code quality, refactoring priorities, or maintenance backlog. (file: r5/tech-debt/SKILL.md)
- engineering:testing-strategy: Design test strategies and test plans. Trigger with "how should we test", "test strategy for", "write tests for", "test plan", "what tests do we need", or when the user needs help with testing approaches, coverage, or test architecture. (file: r5/testing-strategy/SKILL.md)
- github:gh-address-comments: Address actionable GitHub pull request review feedback. Use when the user wants to inspect unresolved review threads, requested changes, or inline review comments on a PR, then implement selected fixes. Use the GitHub app for PR metadata and flat comment reads, and use the bundled GraphQL script via `gh` whe (file: r8/gh-address-comments/SKILL.md)
- github:gh-fix-ci: Use when a user asks to debug or fix failing GitHub PR checks that run in GitHub Actions. Use the GitHub app from this plugin for PR metadata and patch context, and use `gh` for Actions check and log inspection before implementing any approved fix. (file: r8/gh-fix-ci/SKILL.md)
- github:github: Triage and orient GitHub repository, pull request, and issue work through the connected GitHub app. Use when the user asks for general GitHub help, wants PR or issue summaries, or needs repository context before choosing a more specific GitHub workflow. (file: r8/github/SKILL.md)
- github:yeet: Publish local changes to GitHub by confirming scope, committing intentionally, pushing the branch, and opening a draft PR through the GitHub app from this plugin, with `gh` used only as a fallback where connector coverage is insufficient. (file: r8/yeet/SKILL.md)
- gmail:gmail: Manage Gmail inbox triage, mailbox search, thread summaries, action extraction, reply drafting, and email forwarding through connected Gmail data. Use when the user wants to inspect a mailbox or thread, search email with Gmail query syntax, summarize messages, extract decisions and follow-ups, prepare replies (file: r9/gmail/SKILL.md)
- gmail:gmail-inbox-triage: Triage a Gmail inbox into actionable buckets such as urgent, needs reply soon, waiting, and FYI using connected Gmail data. Use when the user asks to triage the inbox, rank what needs attention, find what still needs a reply, or separate important mail from noise. (file: r9/gmail-inbox-triage/SKILL.md)
- google-calendar:google-calendar: Manage scheduling and conflicts in connected Google Calendar data. Use when the user wants to inspect calendars, compare availability, review conflicts, find a meeting room, review event notes or attachments, add or adjust reminders, place temporary

holds, or draft exact create, update, reschedule, or cancel (file: r10/google-calendar/SKILL.md)

- google-calendar:google-calendar-daily-brief: Build polished one-day Google Calendar briefs. Use when the user asks for today, tomorrow, or a specific date summary with an agenda, conflict flags, free windows, remaining-meeting readouts, or a calendar brief, and the Google Calendar connector is available. (file: r10/google-calendar-daily-brief/SKILL.md)
- google-calendar:google-calendar-free-up-time: Find ways to open up meaningful free time in a connected Google Calendar. Use when the user wants to clear up their day, make room for focus time, create a longer uninterrupted block, or see the smallest set of calendar changes that would give time back. (file: r10/google-calendar-free-up-time/SKILL.md)
- google-calendar:google-calendar-group-scheduler: Find and rank good meeting times for multiple people using connected Google Calendar data. Use when the user wants to schedule a group meeting, compare candidate slots across several attendees, find the best compromise time, or add a room check after narrowing the attendee-compatible options. (file: r10/google-calendar-group-scheduler/SKILL.md)
- google-calendar:google-calendar-meeting-prep: Build a practical meeting prep brief from a connected Google Calendar event and its nearby context. Use when the user wants to prepare for an upcoming meeting, understand what to read beforehand, pull in linked notes or docs, or get a concise brief on what the meeting appears to require. (file: r10/google-calendar-meeting-prep/SKILL.md)
- google-drive:google-docs: Connector-first Google Docs creation and editing in local Codex plugin sessions, with direct native create and batchUpdate workflows for simple docs, DOCX-first import for polished deliverables, target-document checks, smart chip and building-block reconstruction, connector-readback verification, and reference (file: r11/google-docs/SKILL.md)
- google-drive:google-drive: Use connected Google Drive as the single endpoint for Drive, Docs, Sheets, and Slides work. Use when the user wants to find, fetch, organize, share, export, copy, or delete Drive files, or summarize and edit Google Docs, Google Sheets, and Google Slides through one unified Google Drive plugin. (file: r11/google-drive/SKILL.md)
- google-drive:google-drive-comments: Write, reply to, and resolve Google Drive comments on Docs, Sheets, Slides, and Drive files with evidence-backed location context. Use when the user asks to leave comments, review a file with comments, respond to comment threads, or resolve Drive comments. (file: r11/google-drive-comments/SKILL.md)
- google-drive:google-sheets: Analyze and edit connected Google Sheets with range precision. Use when the user wants to create Google Sheets, find a spreadsheet, inspect tabs or ranges, search rows, plan formulas, create or repair charts, clean or restructure tables, write concise summaries, or make explicit cell-range updates. (file: r11/google-sheets/SKILL.md)
- google-drive:google-slides: Google Slides work for finding, reading, summarizing, creating, importing, template following, visual cleanup, source-deck adaptation, structural repair, and content edits in native Slides decks. (file: r11/google-slides/SKILL.md)
- pdf:pdf: Read, create, inspect, render, and verify PDF files where visual layout matters. Use Poppler rendering plus Python tools such as reportlab, pdfplumber, and pypdf for generation and extraction. (file: r13/pdf/26.630.12135/skills/pdf/SKILL.md)
- presentations:Presentations: Create or edit PowerPoint or Google Slides decks (file: r13/presentations/26.630.12135/skills/presentations/SKILL.md)
- productivity:memory-management: Two-tier memory system that makes Claude a true workplace collaborator. Decodes shorthand, acronyms, nicknames, and internal language so Claude understands requests like a colleague would. CLAUDE.md for working memory, memory/ directory for the full knowledge base. (file: r6/memory-management/SKILL.md)
- productivity:start: Initialize the productivity system and open the dashboard. Use when setting up the plugin for the first time, bootstrapping working memory from your existing task list, or decoding the shorthand (nicknames, acronyms, project codenames) you use in your todos. (file: r6/start/SKILL.md)
- productivity:task-management: Simple task management using a shared TASKS.md file. Reference this when the user asks about their tasks, wants to add/complete tasks, or needs help tracking commitments. (file: r6/task-management/SKILL.md)
- productivity:update: Sync tasks and refresh memory from your current activity. Use when pulling new assignments from your project tracker into TASKS.md, triaging stale or overdue tasks, filling memory gaps for unknown people or projects, or running a comprehensive scan to catch todos buried in chat and email. (file: r6/update/SKILL.md)
- skill-creator: Create new skills, modify and improve existing skills. Use when users want to create a skill from scratch, edit, or optimize an existing skill. (file: r0/skill-creator/SKILL.md)
- slack:slack: Read Slack context, route to the right Slack workflow, and prepare or perform Slack writes that match the user's intent. (file: r12/slack/SKILL.md)
- slack:slack-channel-summarization: Summarize activity from one Slack channel and return a concise recap, post-ready update, or summary doc. (file: r12/slack-channel-

summarization/SKILL.md)

- slack:slack-daily-digest: Create a daily Slack digest from selected channels or topics. Use when the user asks for a daily Slack recap or summary of today's Slack activity. (file: r12/slack-daily-digest/SKILL.md)
- slack:slack-notification-triage: Triage recent Slack activity into a priority queue or task list for the user. (file: r12/slack-notification-triage/SKILL.md)
- slack:slack-outgoing-message: Primary skill for composing, drafting, or refining any outbound Slack content. Use this whenever the task will require using `slack\_send\_message`, `slack\_send\_message\_draft`, or `slack\_create\_canvas`. Use `slack` to read or analyze Slack context; use this skill to produce the final outgoing message. (file: r12/slack-outgoing-message/SKILL.md)
- slack:slack-reply-drafting: Draft Slack replies from available context. Use when the user wants help finding messages that likely need a response and preparing reply drafts. (file: r12/slack-reply-drafting/SKILL.md)
- spreadsheets:Spreadsheets: Use this skill when a user requests to create, modify, analyze, visualize, or work with spreadsheet files (`.xlsx`, `.xls`, `.csv`, `.tsv`) or Google Sheets-targeted spreadsheet artifacts with formulas, formatting, charts, tables, and recalculation. (file: r13/spreadsheets/26.630.12135/skills/spreadsheets/SKILL.md)
- template-creator:template-creator: Create or update a reusable personal Codex artifact-template skill. Use when the user invokes \$template-creator or asks in natural language to create a template using, from, or based on an attached Word document, PowerPoint presentation, or Excel workbook, or explicitly asks to edit or update a passed arti (file: r13/template-creator/26.630.12135/skills/template-creator/SKILL.md)

How to use skills

- Discovery: The list above is the skills available in this session (name + description + short path). Skill bodies live on disk at the listed paths after expanding the matching alias from `### Skill roots`.
- Trigger rules: If the user names a skill (with `\$SkillName` or plain text) OR the task clearly matches a skill's description shown above, you must use that skill for that turn. Multiple mentions mean use them all. Do not carry skills across turns unless re-mentioned.
- Missing/blocked: If a named skill isn't in the list or the path can't be read, say so briefly and continue with the best fallback.
- How to use a skill (progressive disclosure):
 - 1) After deciding to use a skill, the main agent must expand the listed short `path` with the matching alias from `### Skill roots`, then open and read its `SKILL.md` completely before taking task actions. If a read is truncated or paginated, continue until EOF.
 - 2) When `SKILL.md` references relative paths (e.g., `scripts/foo.py`), resolve them relative to the directory containing that expanded `SKILL.md` first, and only consider other paths if needed.
 - 3) If `SKILL.md` points to extra folders such as `references/`, use its routing instructions to identify the files required for the task. The main agent must read each required instruction or reference file itself before acting on it. Do not delegate reading, summarizing, or interpreting skill instructions to a subagent. Subagents may still perform task work when the selected skill allows it.
 - 4) If `scripts/` exist, prefer running or patching them instead of retyping large code blocks.
 - 5) If `assets/` or templates exist, reuse them instead of recreating from scratch.
- Coordination and sequencing:
 - If multiple skills apply, choose the minimal set that covers the request and state the order you'll use them.
 - Announce which skill(s) you're using and why (one short line). If you skip an obvious skill, say why.
- Context hygiene:
 - Progressive disclosure applies to selecting relevant files, not partially reading a selected instruction file. Do not load unrelated references, scripts, or assets.
 - Avoid deep reference-chasing: prefer opening only files directly linked from `SKILL.md` unless you're blocked.
 - When variants exist (frameworks, providers, domains), pick only the relevant reference file(s) and note that choice.
 - Safety and fallback: If a skill can't be applied cleanly (missing files, unclear instructions), state the issue, pick the next-best approach, and continue.

</skills_instructions>

<plugins_instructions>

Plugins

A plugin is a local bundle of skills, MCP servers, and apps.

How to use plugins

- Skill naming: If a plugin contributes skills, those skill entries are prefixed with ``plugin_name:`` in the Skills list.
 - MCP naming: Plugin-provided MCP tools keep standard MCP identifiers such as ``mcp__server__tool``; use tool provenance to tell which plugin they come from.
 - Trigger rules: If the user explicitly names a plugin, prefer capabilities associated with that plugin for that turn.
 - Relationship to capabilities: Plugins are not invoked directly. Use their underlying skills, MCP tools, and app tools to help solve the task.
 - Relevance: Determine what a plugin can help with from explicit user mention or from the plugin-associated skills, MCP tools, and apps exposed elsewhere in this turn.
 - Missing/blocked: If the user requests a plugin that does not have relevant callable capabilities for the task, say so briefly and continue with the best fallback.
- </plugins_instructions>

2. user (2026-07-05T16:31:47.924Z)

AGENTS.md instructions

<INSTRUCTIONS>

Global instructions (all projects)

Repo-freshness convention: git pull BEFORE reading

****Always `git pull` a repo before starting to read it**** ? at session start for the primary working directory, and again for any sibling/local repo the moment work touches it.

- ****Why:**** the owner closes every session on a "clean slate", but multiple parallel slates (sessions/machines) exist ? PRs get merged and master moves between sessions, so the local checkout can silently lag the remote truth. Pull first so ramp-up reads (handoff, docs, code) reflect where the remote actually is.
- ****How (safe form):**** ``git pull --ff-only`` on the current branch (a plain fetch+ff). Never auto-merge, rebase, or discard local state to make a pull succeed.
- ****If it can't fast-forward**** (diverged branch, dirty tree in the way): do NOT force anything ? report the state to the owner and proceed read-only on what's local, flagging that it may be stale. Uncommitted changes are often a sibling session's or the owner's WIP; never clobber them.

Git branching safety: concurrent sessions / shared checkouts

Multiple sessions ? and spawned/background tasks ? may share ONE working checkout and create branches + commit ****concurrently****, so ``git branch`` / ``git log`` can change UNDER you between two commands. (Spawned "fresh worktree" tasks are NOT always isolated.) This has caused a real incident: a sibling commit landed in the gap between a branch-point check and ``git checkout -b``, so the new branch rode on top of it and a ****squash-merge** silently absorbed the sibling's work******. Protocol ? do this EVERY time, not only when you suspect a collision:

- ****Branch from the REMOTE, explicitly:**** ``git fetch origin`` then ``git checkout -b <name> origin/<base>`` (e.g. ``origin/master``). Never assume the current branch is the intended base.
- ****Re-verify right before `git add`/commit:**** ``git branch --show-current`` + ``git log --oneline -1`` (HEAD is yours). Before opening a PR, ``git log --oneline origin/<base>..HEAD`` must list ONLY your commits ? if a sibling commit appears, re-cut the branch from ``origin/<base>``.
- ****Stage explicit paths**** (``git add <files>``) ? never ``git add -A`` / ``./`` / ``-u``, so sibling or untracked files can't ride into your commit.
- ****Serialize repo writes:**** don't start a repo-writing spawned/background task while you hold uncommitted work ? commit or push first. If one may be running, treat the tree + current branch as able to shift, and put this branching-safety reminder into the spawned task's own prompt.

Session-handoff convention

Maintain a living ****session handoff**** for each project and use it to resume context quickly

across windows.

- **Where it lives:**
 - If the project has an auto-memory dir (`~/Codex/projects/<project>/memory/`), keep the handoff as `session-handoff.md` there, and list it under a **"? Start here"** entry at the top of that project's `MEMORY.md`.
 - Otherwise, keep a `SESSION\_HANDOFF.md` at the repo root.
- **What it contains** (keep it to ~1 page ? it POINTS to detailed artifacts, never duplicates them):
 1. **You are here** ? current state.
 2. **Next steps / open threads.**
 3. **Ramp-up kit** ? the exact files/docs to read to get current.
 4. **Key decisions** ? so they aren't relitigated.
- **At session start:** if a handoff exists, read it before starting substantive work, and use it (plus its ramp-up kit) to get up to speed instead of replaying past conversation.
- **Updating it:** keep it current **automatically** ? refresh at meaningful checkpoints (a PR is pushed/merged, a key decision is made, work shifts phase, or a session is wrapping up), so a restart can ramp up without losing context. Do **not** rewrite it every turn ? only when something worth resuming from changed. The phrases **"update the handoff"** / **"refresh the handoff"** force an immediate full rewrite on demand.
- **Keep it honest:** it reflects real, shipped state ? never aspirational. (See each project's attribution/working-agreement note where present.)

Code review ? pre-LGTM gate (applies to every PR/code review, every project)

Do **not** post **LGTM** or otherwise sign off on a pull request until I have personally **verified** the following ? never trust the PR description's claims, reproduce them locally:

1. **Full test suite is green.** Run the whole suite (e.g. `run\_all\_tests.py` / `pytest`), not just the changed files.
2. **Install any dependency needed** so the suite actually collects and runs ? do not let modules skip or error for missing deps. (If a check genuinely can't run locally ? GPU/WSL/real-video/etc. ? say so explicitly and note what was and wasn't verified.)
3. **Lint + type checks are clean** ? `ruff check .` and `mypy ...`, matching the repo's CI. Install `ruff`/`mypy` if absent.
4. **Coverage stays up** ? run with coverage and confirm it stays **at or above the repo's floor** (e.g. the CI `--cov-fail-under` value) and does **not regress** versus the base branch.
5. **Review findings are resolved** ? bugs and repo-convention/discipline issues addressed.

LGTM is a **verified** stamp, never a rubber stamp. State plainly what was and wasn't verifiable locally.

</INSTRUCTIONS>

<environment_context>

```
<cwd>C:\Users\avidu\Documents\Codex\2026-07-05\can</cwd>
<shell>powershell</shell>
<current_date>2026-07-05</current_date>
<timezone>Asia/Calcutta</timezone>
<filesystem><workspace_roots><root>C:\Users\avidu\Documents\Codex</root><root>C:\Users\avidu\Documents\Codex\2026-07-05\can</root></workspace_roots><permission_profile
type="disabled"><file_system type="unrestricted" /></permission_profile></filesystem>
</environment_context>
```

3. user (2026-07-05T16:31:47.960Z)

Is it possible to get a markdown/pdf collection of all the Codex coding sessions and be able to archive them for my personal use locally and then delete the sessions in Codex?

4. assistant (2026-07-05T16:32:02.063Z)

I'll check the local Codex capabilities first, because this is one of those ?possible, but the exact path depends on what this app exposes? questions. Then I'll give you the cleanest export/archive/delete workflow rather than hand-waving.

5. assistant (2026-07-05T16:32:05.766Z)

I found local tools for listing Codex threads and archiving them, but not an exposed `?delete thread?` tool in this session. I'm going to check the Codex manual next so I don't overclaim what's supported versus what we can script around locally.

6. assistant (2026-07-05T16:32:15.972Z)

The manual is fresh locally. I'm checking the app and storage sections now, because the useful answer here hinges on whether Codex has a first-class export/delete flow or whether this needs a small local archival script.

7. assistant (2026-07-05T16:32:23.567Z)

The manual does document CLI-level ``archive``, ``unarchive``, and ``delete`` support, plus app-server thread APIs for creating/listing/archiving conversations. I'm checking whether the local CLI exposes enough to export content directly or whether we need to derive Markdown from the local session files.

8. assistant (2026-07-05T16:32:29.141Z)

The bundled ``codex.exe`` is present but Windows is denying direct execution from this shell, so I can't validate CLI help here. The app connector is callable, though, so I'm going to sample the thread metadata shape; that tells us what can be automated safely from inside this session.

9. assistant (2026-07-05T16:32:43.750Z)

The thread list exposes IDs, titles, previews, timestamps, status, and workspace paths. That is enough for an index, but not enough by itself for a full Markdown/PDF transcript, so I'm checking local Codex storage next.

10. assistant (2026-07-05T16:32:54.383Z)

There is a local ``~/codex/sessions`` directory and SQLite state/log databases, so a local archive is feasible in principle. I'm peeking at file names and formats only, not dumping transcript contents into chat.

11. assistant (2026-07-05T16:33:20.697Z)

Yes. I confirmed your local Codex has session data under ``C:\Users\avidu\codex\sessions`` and already-archived session data under ``C:\Users\avidu\codex\archived_sessions``, so a local Markdown/PDF archive is very doable.

The clean workflow would be:

1. Export every session to a local archive:
 - one Markdown file per thread
 - optional one PDF per thread
 - an index file with title, date, thread id, project path, status
 - raw JSONL backups plus hashes, so the archive is verifiable
2. Verify the archive:
 - spot-check a few sessions
 - confirm PDFs open
 - confirm the number of exported sessions matches Codex's thread list
3. Archive sessions in Codex:
 - this session has a callable Codex archive tool, so I can archive threads from here after confirmation.
4. Delete sessions:
 - the current Codex manual documents CLI support for ``codex delete``, but my shell cannot run the bundled WindowsApps ``codex.exe`` directly right now due to Windows access denial.
 - deletion may need the Codex UI or a working CLI path. I would only do it after export

verification and an explicit final confirmation.

My recommendation: export first, archive second, delete last. Nice and boring, which is exactly what you want for preserving personal work history.

12. user (2026-07-05T16:38:41.464Z)

Sounds good, maybe lets go a step further and make it a private github repo as well so I can add sessions from my other computers and other agents? I made a "C:\Users\avidu\Projects\Agent Sessions" directory for this work. Can you incorporate this and if you could find the Claude and VSCode? I have Z.AI and Deepseek V4 and grok as well who should store sessions locally as well and Gemini but it may not have local unless you find a way to fetch from Google servers for which I can give you any access!

13. assistant (2026-07-05T16:38:59.395Z)

I'll treat `C:\Users\avidu\Projects\Agent Sessions` as the archive project. First I'm going to check whether it's already a git repo, fast-forward it if it is, then map the local session stores for Codex, Claude, VS Code-style chat logs, and the other agents before I create any archive tooling.

14. assistant (2026-07-05T16:39:09.257Z)

I'm using the GitHub workflow skill for the repo/publishing side, and I'll keep the first pass local and auditable. The archive repo should contain tools and exported artifacts, but no automatic deletion or cloud scraping until you explicitly bless that part.

15. assistant (2026-07-05T16:39:23.037Z)

`Agent Sessions` is empty and not a git repo yet. `gh` is installed and authenticated as `avidullu`, so creating a private GitHub repo from this machine is available once the local archive project has a sensible first commit.

16. assistant (2026-07-05T16:39:55.405Z)

I found likely stores for `~\.claude`, `~\.gemini`, VS Code global/workspace storage, GitHub Copilot Chat, and a DeepSeek V4 VS Code extension. I don't see obvious local Grok or Z.AI stores in the standard Windows locations yet, so I'm widening that discovery just enough to catch extension/app folders without crawling the whole disk.

17. user (2026-07-05T16:40:41.823Z)

For Grok you may want to check wsl install

18. assistant (2026-07-05T16:40:49.358Z)

Good catch. I'll include WSL in the discovery pass for Grok/xAI and any Linux-side agent CLIs; that's a very plausible place for their session caches to be hiding.

19. assistant (2026-07-05T16:41:31.830Z)

WSL turned out to matter: Ubuntu has `~/.grok`, Grok completions, a Grok binary, Grok session data, plus WSL-side Claude/Codex/Gemini and a ZAI VS Code server extension. I'm going to make the repo's discovery/import config handle both Windows paths and WSL paths so the other machines can feed it the same way.

20. assistant (2026-07-05T16:41:48.486Z)

The WSL Grok install has an actual `~/.grok/sessions` area, but the first broad listing was too noisy because the install also carries plugins, docs, binaries, and cached marketplace content. I'm narrowing to transcript-like files and session databases now.

21. assistant (2026-07-05T16:41:57.177Z)

Discovery is enough to start building: Codex and Claude have JSONL transcript stores; Gemini Antigravity has local transcript JSONL; Grok has WSL `chat\_history.jsonl` plus a `grok sessions list`; VS Code has Copilot/DeepSeek/ZAI storage but Copilot appears mostly state/cache on this machine. I'm going to scaffold the archive repo with a source inventory, import config, and a converter that can produce Markdown now and PDFs when a renderer is installed.

22. assistant (2026-07-05T16:42:24.044Z)

Before writing the converters I'm checking the event shapes, but only structurally: keys, roles, and string lengths, not dumping private transcript text into chat. That lets the parser be useful across Codex, Claude, Gemini, and Grok without hardcoding one brittle format.

23. assistant (2026-07-05T16:42:49.131Z)

The formats are friendly enough: Codex and Claude are JSONL with explicit message records, Gemini Antigravity logs have content-bearing steps, and Grok's WSL `chat\_history.jsonl` has compact role/content records. I'm initializing the archive repo now and adding a first-pass importer around those shapes.

24. assistant (2026-07-05T16:43:10.979Z)

I'm writing this as a normal, portable Python tool with no service-specific secrets in config. It will default to local discovery paths, emit Markdown, optionally emit simple PDFs if `reportlab` is installed, and keep raw-log backup opt-in so the Git repo does not balloon by surprise.

25. assistant (2026-07-05T16:45:01.991Z)

The repo scaffold and importer are in place. I'm running a small smoke export first, then I'll run discovery and the real Codex export so we can catch parser mistakes before creating the private remote.

26. assistant (2026-07-05T16:45:13.062Z)

Smoke export succeeded for two Codex files. I'm checking the generated index and a snippet of the Markdown structure now, then I'll expand the run.

27. assistant (2026-07-05T16:45:40.467Z)

I found one link bug in the generated archive index: the Markdown links were written as if the index lived at repo root, but the index itself lives inside `archive/`. I'm fixing that and adding the WSL/Vs Code discovery roots we found so the report tells the whole story.

28. assistant (2026-07-05T16:46:07.618Z)

The tool now knows about WSL Claude, WSL DeepSeek request dumps, Copilot Chat inventory on Windows/WSL, and the WSL ZAI extension location. I'm mirroring those in the example config so future machines can copy it cleanly.

29. assistant (2026-07-05T16:46:24.131Z)

I'm doing a quick syntax check and regenerating discovery. After that I'll run the full Markdown export, then decide whether local PDF generation is sane for the whole corpus.

30. assistant (2026-07-05T16:46:35.560Z)

`reportlab` is already available, which is nice. I'll export the full Markdown archive first; it's the durable source-of-truth format. Then I'll generate PDFs for the main transcript sources rather than blindly PDF-ing every DeepSeek request dump unless the size looks reasonable.

31. assistant (2026-07-05T16:47:56.975Z)

Full Markdown export completed: 3,114 session/request files. I'm checking size and source

breakdown before PDF-ing everything, because PDFs can multiply repository weight pretty quickly.

32. assistant (2026-07-05T16:48:05.566Z)

The full export surfaced a real archival issue: some source files collide after slugging, so the exporter counted 3,114 inputs but wrote fewer Markdown files. I'm adding a hash suffix to every exported filename and regenerating the archive so each source file maps to a unique artifact.

33. assistant (2026-07-05T16:48:11.750Z)

I'm clearing only the generated `archive` contents and rerunning the exporter with the new unique-name rule. The source stores stay untouched.